

Entregável — Exercício 3.2

Projeto: NovaTech Assistant API | Revisão Crítica de Outputs de IA

1. Minha Revisão Manual

Antes de utilizar a IA, identifiquei os seguintes problemas no código gerado pelo Copilot:

- **Uso de `as any` sem validação Zod:** Violação do `AGENTS.md` e risco de bug por falta de validação de input.
- **Logging de PII:** O código incluía o `attendantEmail` no objeto de log em texto plano, causando problema de segurança e violação de regras.
- **Uso de `console.log`:** Violação do padrão de observabilidade exigido (Pino logger).
- **Require Dinâmico:** A importação do cliente Cosmos no corpo do handler, o que viola o `AGENTS.md` e prejudica a performance por não reaproveitar a conexão.

2. Revisão do Claude

A revisão automatizada apontou os seguintes pontos:

- **[SEC-1] Crítico:** PII sendo logado (email do atendente) violando as diretrizes.
- **[SEC-2] Crítico:** `require()` dinâmico que compromete performance no Azure Functions.
- **[BUG-1] Crítico:** Ausência total de validação de input via Zod.
- **[LOG-1] Alto:** Utilização de `console.log` em detrimento do logger Pino estruturado.
- **[BUG-2] Alto:** Falta de tratamento de erro (ausência de `try/catch` ao invocar o banco de dados).
- **[TYPE-1] Médio:** Tipagem insegura ao utilizar `as any` no parse inicial do body HTTP.

3. Comparação das Revisões

Ambas as análises alinham-se na identificação das infrações mais graves do `AGENTS.md`: a exposição de dados sensíveis (PII), a ausência de tipagem e validação forte (Zod), a prática inadequada de dependências em tempo de execução (require dinâmico) e a falha de observabilidade (console.log).

Como distinção técnica: a revisão do Claude foi além das regras de negócio/segurança e salientou uma vulnerabilidade operacional crítica (falta de um bloco `try/catch` que provocaria erros HTTP 500 não tratados em caso de falha no banco de dados). Isto demonstra como a revisão cruzada gera um resultado muito mais sólido.

4. Código Refatorado

O módulo final foi reescrito para cumprir estritamente as regras de arquitetura e segurança, mascarando dados sensíveis e gerindo erros de forma consistente:

```
import { app, HttpRequest, HttpResponseInit } from '@azure/functions';
import { CosmosClient } from '@azure/cosmos';
import { z } from 'zod';
import { logger } from '../shared/logger';

// Schema de validação com Zod
const FeedbackSchema = z.object({
  queryId: z.string().min(1, 'queryId é obrigatório'),
  rating: z.number().int().min(1).max(5, 'rating deve estar entre 1 e 5'),
  comment: z.string().min(1, 'comment é obrigatório'),
  attendantEmail: z.string().email('attendantEmail inválido')
});

type Feedback = z.infer<FeedbackSchema>;

// Inicializar cliente Cosmos uma única vez (reaproveitamento de conexão)
const cosmosClient = new CosmosClient(process.env.COSMOS_CONNECTION_STRING!);
const database = cosmosClient.database('novatech');
const container = database.container('feedbacks');

// Função auxiliar para mascarar email nos logs
function maskEmail(email: string): string {
  const [localPart, domain] = email.split('@');
  const masked = localPart.substring(0, 2) + '*'.repeat(Math.max(0, localPart.length - 2));
  return `${masked}@${domain}`;
}

export async function feedbackHandler(
  request: HttpRequest
): Promise<HttpResponseInit> {
  try {
    const body = await request.json();

    // Validar o body com Zod
    const validatedFeedback = FeedbackSchema.parse(body);

    const feedback = {
      ...validatedFeedback,
      timestamp: new Date().toISOString()
    };

    // Log com email mascarado
    const logData = {
      ...feedback,
      attendantEmail: maskEmail(feedback.attendantEmail)
    };

    logger.info('Feedback recebido', logData);

    // Persistir no Cosmos DB
    await container.items.create(feedback);

    return { status: 200 };
  } catch (error) {
```

```
if (error instanceof z.ZodError) {
  logger.warn('Validação de feedback falhou', { errors: error.errors });
  return {
    status: 400,
    jsonBody: { error: 'Dados inválidos', details: error.errors }
  };
}

logger.error('Erro ao processar feedback', {
  error: error instanceof Error ? error.message : String(error)
});
return { status: 500 };
}
}

app.http('feedback', {
  methods: ['POST'],
  handler: feedbackHandler
});
```